

Praktikum „Bildverarbeitung“ (WS 2025/26 – Dr.-Ing. Lorenz Fieten) – 3. Aufgabe

Die Eigenschaften von Filtern können u. a. anhand von synthetisch erzeugten Testbildern untersucht werden. In der Software-Entwicklung können derartige Testbilder auch nützlich sein, um entwickelte Funktionalität mit automatisierten Tests abzusichern.

Testbilder mit konzentrischen Kreisen, deren Mittelpunkt jeweils in der Bildmitte liegt, können insbesondere dann verwendet werden, wenn es um die Isotropie-Eigenschaften von Glättungsfiltern geht. Ein Beispiel für ein derartiges Ringtestbild ist die aus der Optik bekannte Fresnel-Zonenplatte. Die darin dargestellten Strukturen umfassen alle Richtungen und alle Wellenzahlen bis zu einer wählbaren maximalen Wellenzahl [Jähne 2024]. Daher eignet sich die Fresnel-Zonenplatte insbesondere zur Analyse von Filter-Operationen auf hoch- und niederfrequente Bildsignalanteile.

Ringtestbilder können ebenfalls eingesetzt werden, um Kantenfilter oder Rangordnungsfilter wie Maximum- und Minimum-Filter zu untersuchen.

Im Rahmen dieser Aufgabe sollen wesentliche Aspekte der Analyse von Filtereigenschaften auf Basis von synthetisch erzeugten Testbildern anhand eines einfachen Beispiels beleuchtet werden.

Dabei sollen insbesondere die folgenden Lernziele erreicht werden:

- Sie sind in der Lage, ein Ringtestbild auf Basis einer formelmäßigen Beschreibung zu erzeugen.
- Sie sind in der Lage, ein Ringtestbild vor und nach der Filterung zu interpretieren.
- Sie sind in der Lage, den Effekt der Filterung anschaulich darzustellen.
- Sie wissen, wie Sie mittels OpenCV Transformationsmatrizen für affine Transformationen berechnen und Bilder entsprechend (geometrisch) transformieren können.
- Sie kennen verschiedene von OpenCV zur Verfügung gestellte Möglichkeiten zur Behandlung von Bildrandbereichen und können beurteilen, welche dieser Möglichkeiten im speziellen Anwendungskontext sinnvoll ist.
- Sie kennen verschiedene von OpenCV zur Verfügung gestellte Möglichkeiten zur Interpolation von Pixel-Werten und können beurteilen, welche dieser Möglichkeiten im speziellen Anwendungskontext sinnvoll ist.
- Sie wissen, wie Sie mittels OpenCV Bildinhalte in rechteckigen Regionen effizient manipulieren können.
- Sie kennen verschiedene von OpenCV zur Verfügung gestellte Glättungsfilter und können diese anwenden.
- Sie kennen verschiedene von OpenCV zur Verfügung gestellte Kanten-Detektoren und können diese anwenden.
- Sie sind in der Lage, mittels OpenCV Rangordnungsfilter auf Bilder anzuwenden.

- Vorbereitung -

1. Legen Sie ein neues Projekt an (Konsolen-App): z. B. „ConsoleApplication_Task3“.
2. Erstellen Sie ein lokales Git-Repository.
3. Setzen Sie die Projekt-Eigenschaften so, dass OpenCV benutzt werden kann.
 - a. Für alle Konfigurationen unter C/C++ -> Allgemein
„Zusätzliche Includeverzeichnisse“: C:\opencv\build\include
 - b. Für alle Konfigurationen unter Linker -> Allgemein
„Zusätzliche Bibliotheksverzeichnisse“: C:\opencv\build\x64\vc16\lib
 - c. Für Debug-Konfiguration unter Linker -> Eingabe
„Zusätzliche Abhängigkeiten“: opencv_world4120d.lib
 - d. Für Release-Konfiguration unter Linker -> Eingabe
„Zusätzliche Abhängigkeiten“: opencv_world4120.lib eintragen
4. Fügen Sie oberhalb von `int main()` `#include „opencv2/opencv.hpp“` ein.
5. Legen Sie innerhalb von `int main()` ein Objekt vom Typ `cv::Mat` mit dem Namen `testImage` an.
6. Sobald sich der Code kompilieren lässt, committen Sie Ihren Code.

- Erzeugung eines konfigurierbaren Ringtestbilds -

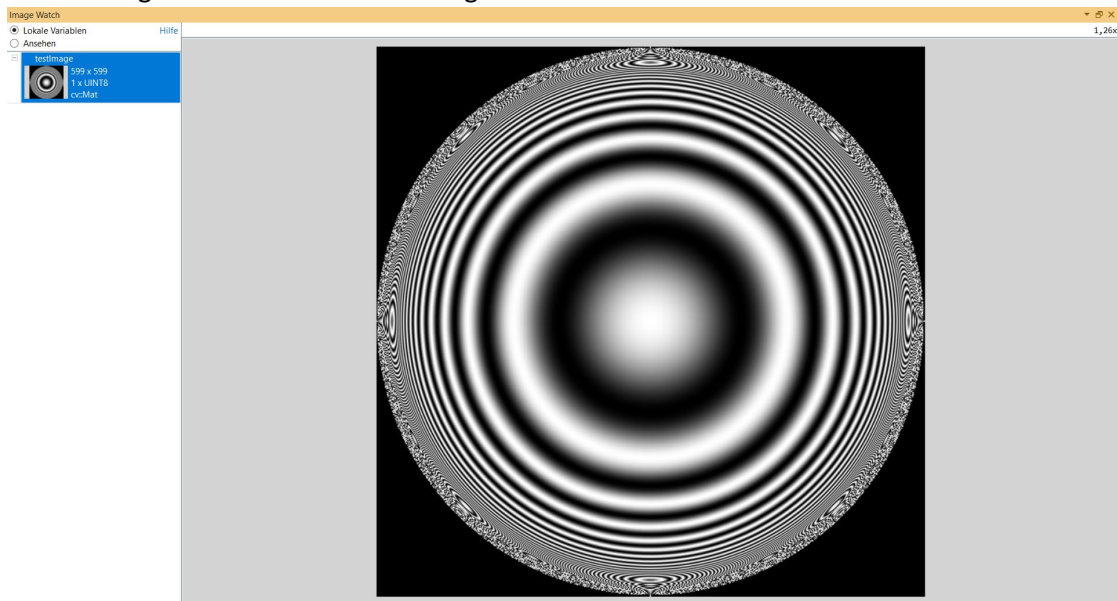
7. Schreiben Sie eine Funktion `cv::Mat getTestImageWithConcentricCircles(int sideLength, int minWavelength, int maxWavelength)`, die ein Ringtestbild durch geeignete Integration der folgenden Code-Zeilen berechnet:

```
const auto dy = testImageCenterY - y;  
const auto dx = testImageCenterX - x;  
const auto dr = sqrt(dx * dx + dy * dy);  
if (dr > (sideLength - 1) / 2.)  
    testImage.at<unsigned char>(y, x) = 0;  
else  
{  
    const auto interpolatedWavelength = maxWavelength - (maxWavelength -  
minWavelength) * dr / ((sideLength - 1) / 2.);  
    const auto pixelValue = (unsigned char)(127.5 * (1. + cos(2. * pi * dr /  
interpolatedWavelength)));  
    testImage.at<unsigned char>(y, x) = pixelValue;  
}
```
8. Führen Sie zwei globale Variablen wie folgt ein:

```
int g_minWavelength = 1;  
int g_maxWavelength = 299;
```
9. Führen Sie zu Beginn der `main`-Funktion eine Variable zur Definition der Bildgröße wie folgt ein:

```
const int sideLength = g_maxWavelength * 2 + 1;
```

10. Berechnen Sie testImage mit der o. g. Funktion und den eingeführten 3 Variablen. Betrachten Sie testImage zur Kontrolle mittels Image Watch. Das Bild sollte so aussehen:



11. Sobald das Ergebnis korrekt aussieht, committen Sie Ihren Code.

- Interaktive Konfiguration des Ringtestbilds und Anzeige in der GUI -

12. Erzeugen Sie innerhalb der main-Funktion ein Fenster wie folgt:
`const std::string resultImageWindowTitle = "Result image";
cv::namedWindow(resultImageWindowTitle, cv::WINDOW_AUTOSIZE);`
13. Unterdrücken Sie störende Ausgaben auf der Konsole durch Einfügen der folgenden Code-Zeile:
`cv::utils::logging::setLogLevel(cv::utils::logging::LOG_LEVEL_WARNING);`
14. Zeigen Sie das „Result image“-Fenster an.
15. Fügen Sie eine Funktion `static void onMinWavelengthTrackbar(int pos, void*)` mit dem folgenden Verhalten hinzu:
a) Beim Aufruf wird `g_minWavelength` auf den Wert von `pos` gesetzt.
b) Ist `pos > g_maxWavelength`, dann wird auch `g_maxWavelength` auf den Wert von `pos` gesetzt.
c) Änderungen von `g_minWavelength` und `g_maxWavelength` werden auf der Konsole ausgegeben.
16. Fügen Sie analog dazu eine Funktion `static void onMaxWavelengthTrackbar(int pos, void*)` mit dem folgenden Verhalten hinzu:
a) Beim Aufruf wird `g_maxWavelength` auf den Wert von `pos` gesetzt
b) Ist `pos < g_minWavelength`, dann wird auch `g_minWavelength` auf den Wert von `pos` gesetzt
c) Änderungen von `g_minWavelength` und `g_maxWavelength` werden auf der Konsole ausgegeben

17. Fügen Sie dem „Result image“-Fenster zwei Trackbars wie folgt hinzu:
`cv::createTrackbar("min wavelength", resultImageWindowTitle, nullptr, g_maxWavelength, onMinWavelengthTrackbar);`
`cv::createTrackbar("max wavelength", resultImageWindowTitle, nullptr, g_maxWavelength, onMaxWavelengthTrackbar);`
18. Setzen Sie für beide Trackbars den kleinsten einstellbaren Wert auf 1.
19. Aktualisieren Sie die im „Result image“-Fenster angezeigten Trackbar-Positionen durch die Verwendung einer Endlosschleife, die beim Drücken der Escape-Taste beendet wird
20. Ermöglichen Sie ein Schließen des „Result image“-Fensters auch durch Klicken auf „x“ wie folgt:
 - a) Fügen Sie `#include <Windows.h>` und `#include <AtlBase.h>` ein
 - b) Ergänzen Sie an passender Stelle `const std::wstring resultImageWindowTitleW = CA2W(resultImageWindowTitle.c_str()).m_psz;`
 - c) Ersetzen Sie `while (true)` durch `while(FindWindow(NULL, resultImageWindowTitleW.c_str()))`

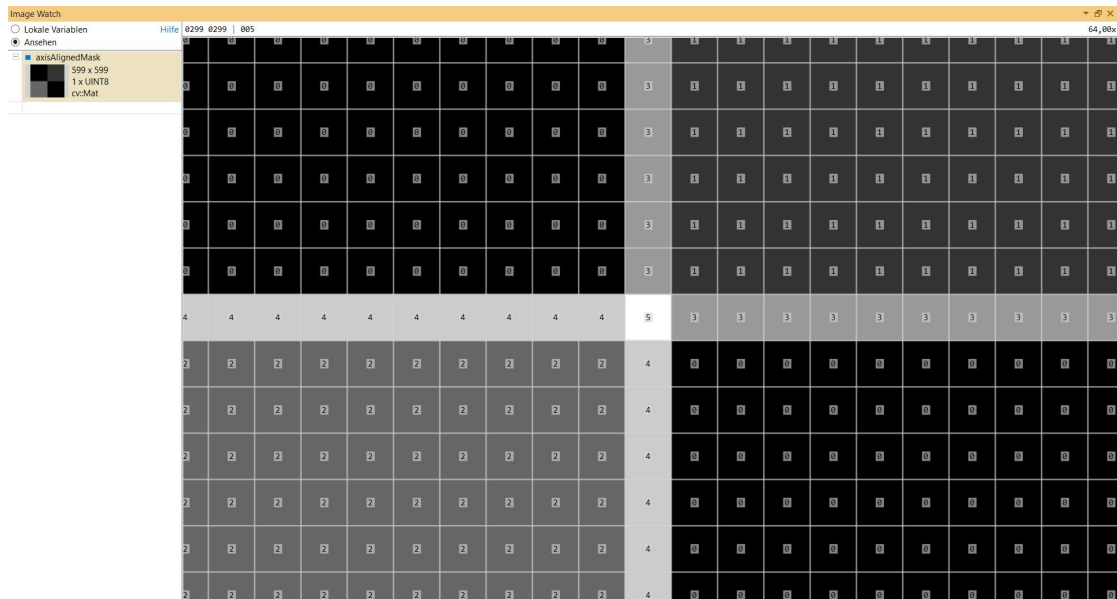
Hintergrundinformationen dazu finden Sie bei Interesse hier:
<https://forum.opencv.org/t/official-ocv-4-replacement-for-cvgetwindowhandle/4535/6>
21. Zeigen Sie im „Result image“-Fenster das Ringtestbild, welches zu den aktuellen Trackbar-Einstellungen passt, an.
22. Sobald das gewünschte Verhalten erhalten wird, committen Sie Ihren Code.

- Berechnung zweier unterschiedlicher gefilterter Bilder -

23. Erzeugen Sie zum aktuellen Ringtestbild ein gefiltertes Bilder `cv::Mat firstProcessedTestImage` durch Anwendung eines 5x5-Box-Filters (mit OpenCV-Standard-Einstellungen)
24. Erzeugen Sie zum aktuellen Ringtestbild ein gefiltertes Bilder `cv::Mat secondProcessedTestImage` durch Anwendung eines 9x9-Box-Filters (mit OpenCV-Standard-Einstellungen)
25. Betrachten Sie beide Bilder mittels Image Watch. Was können Sie beobachten?
26. Sobald das gewünschte Verhalten erhalten wird, committen Sie Ihren Code.

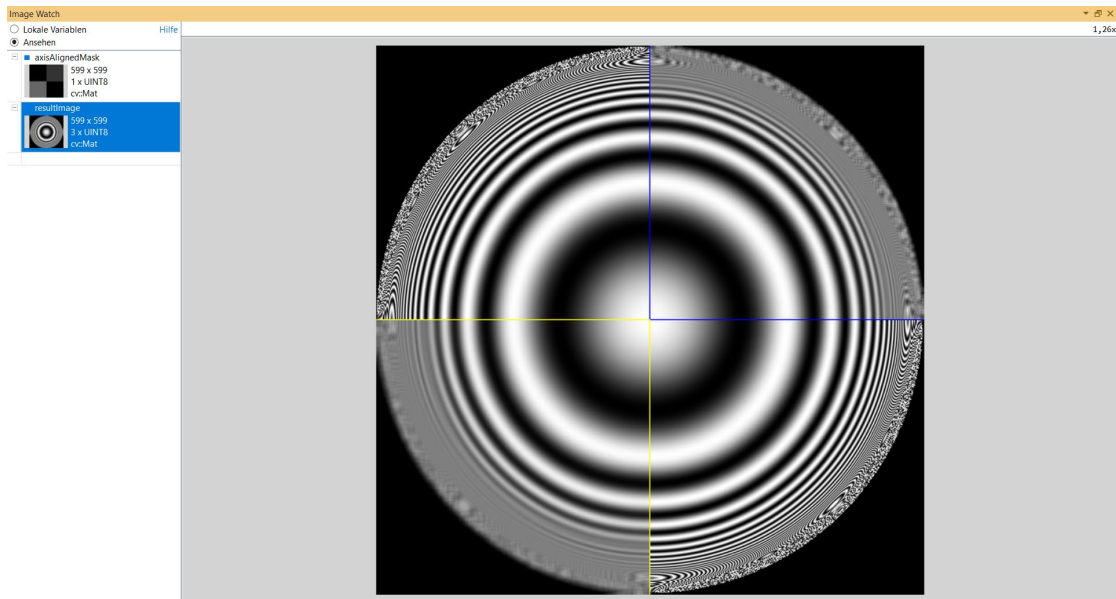
- Erzeugung eines partitionierten Ergebnisbilds aus beiden gefilterten Bildern und dem Ringtestbild -

27. Schreiben Sie eine Funktion `cv::Mat getAxisAlignedMask(int sideLength)`, die das folgende in Image Watch dargestellte Bild, welches aus Blöcken homogener Grauwerte besteht, generiert:



Benutzen Sie hierzu nur `cv::Mat::zeros(...)`, `cv::Range(...)` und `setTo(...)`.
(Das kann übersichtlich mit 9 Zeilen Code realisiert werden.)

28. Erzeugen Sie durch den Einsatz der neuen Funktion (außerhalb der o. g. while-Schleife) eine `cv::Mat axisAlignedMask` und kontrollieren Sie mittels Image Watch Ihr Ergebnis.
29. Schreiben Sie eine Funktion `cv::Mat getResultImage(cv::Mat testImage, cv::Mat firstProcessedTestImage, cv::Mat secondProcessedTestImage, cv::Mat mask)`, die ein partitioniertes Ergebnisbild mit den folgenden Eigenschaften erzeugt:
 - a) Der Bildinhalt von `testImage` soll übernommen werden, wo in `mask` eine 0 steht.
 - b) Der Bildinhalt von `firstProcessedTestImage` soll übernommen werden, wo in `mask` eine 1 steht.
 - c) Der Bildinhalt von `secondProcessedTestImage` soll übernommen werden, wo in `mask` eine 2 steht.
 - d) Das Bild soll gelb sein, wo in `mask` eine 3 steht.
 - e) Das Bild soll blau sein, wo in `mask` eine 4 steht.
 - f) Das Bild soll weiß sein, wo in `mask` eine 5 steht.
 Benutzen Sie hierzu die Funktionen `cv::Mat::ones(...)`, `cv::bitwise_and(...)`, `cv::cvtColor(...)` und `setTo(...)`.
30. Erzeugen Sie durch den Einsatz der neuen Funktion (innerhalb der o. g. while-Schleife) eine `cv::Mat resultImage`. Betrachten Sie `resultImage` zur Kontrolle mittels Image Watch. Das Bild sollte mit den ursprünglichen Tracker-Positionen (`g_minWavelength = 1` und `g_maxWavelength = 299`) so aussehen:



31. Zeigen Sie im „Result image“-Fenster das partitionierte Ergebnisbild, welches zu den aktuellen Trackbar-Einstellungen passt, an.
32. Sobald das gewünschte Verhalten erhalten wird, committen Sie Ihren Code.

- Interaktive Variation der Ergebnisbild-Partitionierung -

33. Führen Sie eine globale Variable wie folgt ein:
`int g_rotationAngleDeg = 0;`
34. Fügen Sie eine Funktion `static void onRotationAngleDegTrackbar(int pos, void*)` mit dem folgenden Verhalten hinzu:
 - a) Beim Aufruf wird `g_rotationAngleDeg` auf den Wert von `pos` gesetzt.
 - b) Änderungen von `g_rotationAngleDeg` werden auf der Konsole ausgegeben.
35. Fügen Sie dem „Result image“-Fenster einen Trackbar wie folgt hinzu:
`cv::createTrackbar("rotation angle in degrees", resultImageWindowTitle, nullptr, 90, onRotationAngleDegTrackbar);`
36. Fügen Sie eine Funktion `cv::Mat getMask(cv::Mat axisAlignedMask, int rotationAngleDeg)`, die eine Rotation der `axisAlignedMask` um den Bildmittelpunkt ermöglicht, hinzu. Benutzen Sie hierfür die Funktionen `cv::getRotationMatrix2D(...)` und `cv::warpAffine(...)`. Wählen Sie dabei unter den von OpenCV zur Verfügung gestellten Optionen ein im spezifischen Anwendungskontext sinnvolles Interpolationsverfahren sowie eine im spezifischen Anwendungskontext sinnvolle Bildrandbehandlung aus.
37. Erzeugen Sie durch den Einsatz der neuen Funktion (innerhalb der o. g. while-Schleife) eine `cv::Mat mask` und kontrollieren Sie mittels Image Watch Ihr Ergebnis (insb. für Rotationswinkel $> 0^\circ$).

38. Verwenden Sie die `cv::Mat mask` (anstelle der `cv::Mat axisAlignedMask`) beim Aufruf der Funktion `getResultImage(...)`, und testen Sie Ihr Ergebnis.

39. Sobald das gewünschte Verhalten erhalten wird, committen Sie Ihren Code.

- Untersuchung anderer Filter -

40. Stellen Sie mit der neuen Anwendung die Filterergebnisse, die mit einem 5x5-Gauß-Filter erhalten werden, denen, die mit einem 9x9-Gauß-Filter erhalten werden, gegenüber. Sie können hierbei aus der Vorlesung bekannte Standard-Werte für die Varianz verwenden.

41. Stellen Sie mit der neuen Anwendung die Filterergebnisse, die mit einem (3x3-)Sobel-Filter erhalten werden, denen, die mit einem Scharr-Filter erhalten werden, gegenüber.

42. Stellen Sie mit der neuen Anwendung die Filterergebnisse, die mit dem Canny-Filter (und Standard-Einstellungen) erhalten werden, denen, die mit einem Laplacian-Filter (und Standard-Einstellungen) erhalten werden, gegenüber.

Beachten Sie hierzu <https://opencv.org/blog/edge-detection-using-opencv/>

43. Stellen Sie mit der neuen Anwendung die Filterergebnisse, die mit einem 5x5-Median-Filter erhalten werden, denen, die mit einem 9x9-Median-Filter erhalten werden, gegenüber.

44. Stellen Sie mit der neuen Anwendung die Filterergebnisse, die mit einer morphologischen Grauwert-Öffnung (mit einer 7x7-Filtermaske) erhalten werden, denen, die mit einer morphologischen Grauwert-Schließung (mit einer 7x7-Filtermaske) erhalten werden, gegenüber.

- Untersuchung der Auswirkung von Bildrauschen -

45. Wiederholen Sie die bisherigen Tests, nachdem Sie das Testbild mit additivem Rauschen gestört haben. Es kann auch – gerade für die Prüfungsvorbereitung – sinnvoll sein, z. B. ein Median-Filter einem Box-Filter gegenüberzustellen.

Melden Sie sich gerne bei Unklarheiten.